



LIONSDEV

Curso de Programação

MÓDULO 9

Autenticação, JWT e MVC



Professor Nicolas Cardoso Motta

Revisando o Módulo 8



MongoDB

Documentos em coleções, no Atlas; a conexão fica no `.env`, em `MONGO_URI`.



Mongoose

O schema define campos e regras; o model faz `create`, `find`, `findById`...



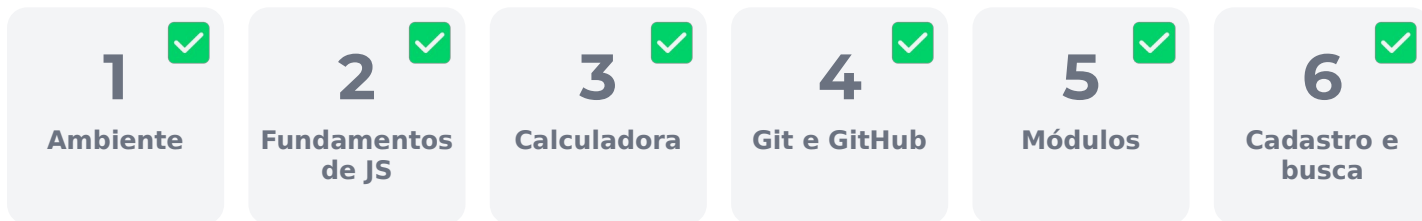
`async/await`

Toda operação no banco é assíncrona: `await` dentro de `try/catch`.

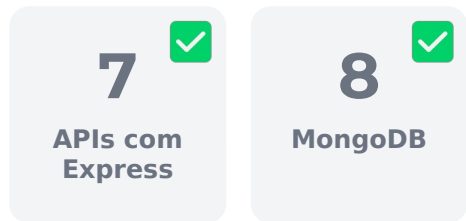
Pergunta para abrir: o que acontece se você esquecer o `await` antes do `Baralho.find()`?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND

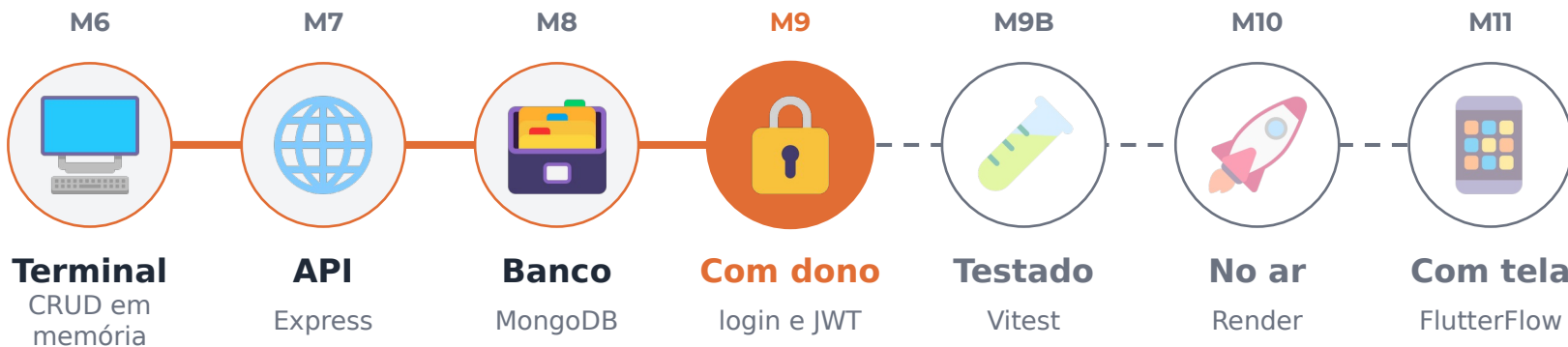


FRONTEND E PROJETO



Você está aqui: **Módulo 9, Autenticação, JWT e MVC**

O projeto flashcards



NO MÓDULO 9

O flashcards entra no boilerplate e ganha dono: cada baralho pertence a um usuário, e só quem está logado vê os próprios baralhos. É o desafio do fim da aula.



Objetivo

- **Organizar a API em camadas, a partir do boilerplate da Lions Dev.**
- **Guardar senhas com segurança usando bcrypt.**
- **Autenticar usuários com JWT e proteger rotas com middleware.**
- **Tratar validação e erros em middlewares próprios.**
- **Controlar permissões por papel (admin x usuário) e trocar a senha com segurança.**
- **Criar um recurso novo, protegido e com dono: os livros.**

**Até agora, a API inteira cabia num
server.js e qualquer pessoa acessava
qualquer rota. Num projeto de
verdade, as duas coisas precisam
mudar.**



Boilerplate

Uma base organizada em camadas

Da organização atual ao boilerplate

Tudo no server.js

- Rotas, regras e acesso ao banco misturados.
- Difícil de testar, manter e dividir em equipe.

Camadas

- Cada arquivo com uma responsabilidade.
- O projeto cresce sem perder a clareza.

O boilerplate é o ponto de partida pronto: você só adiciona os recursos do seu projeto.

As camadas e o que cada uma faz

Camada	Responsabilidade
routes	Liga o método e o caminho ao controller certo
middlewares	Autenticação, validação e tratamento de erros
controllers	Lê o req, chama o service e monta o res
services	As regras de negócio
repositories	Conversa com o banco, pelo Model
models	O Schema do Mongoose
config / utils	Conexão com o banco e funções de apoio

O caminho de uma requisição



Rota

Qual controller atende.

Middleware

Está logado?
Os dados vieram?

Controller

Lê o req e responde.

Service

Aplica as regras.

Repository

Lê e grava no banco.

O resultado volta pelo mesmo caminho até virar a resposta HTTP.

Estrutura de pastas

```
src/
├── app.js           (Express, middlewares, rotas)
├── server.js       (conecta o banco e sobe)
├── config/database.js
├── routes/
├── middlewares/
├── controllers/
├── services/
├── repositories/
├── models/
└── utils/
```

nomes → um arquivo por recurso em cada camada:
livro.routes.js,
livro.service.js...

ES Modules → o boilerplate usa "type": "module".

app.js x server.js

app.js: monta o Express

```
// + imports: express, rotas e erro

const app = express()
app.use(express.json())
app.use('/api/auth', authRoutes)
app.use('/api/usuarios', usuarioRoutes)
app.use(erroMiddleware)

export default app
```

server.js: sobe a aplicação

```
import dotenv from 'dotenv'
import app from './app.js'
import conectarBanco from './config/database.js'

dotenv.config()
await conectarBanco()
app.listen(process.env.PORT || 3000)
```

Separar os dois deixa o app testável sem subir servidor. E o await garante que a API só abre depois de o banco conectar.

Route e controller

```
// routes/agendamento.routes.js
const router = Router()
router.post(
  '/api/agendamento/cadastro',
  AgendamentoController.cadastrar
)

// controllers/agendamento.controller.js
async function cadastrar(req, res, next) {
  try {
    const novo = await AgendamentoService.cadastrar(req.body)
    res.status(201).json({ agendamento: novo })
  } catch (erro) {
    next(erro)
  }
}
```

A rota só aponta o controller. O controller é o único que conhece req e res: lê, delega ao service e responde.

Service e repository

```
// services/agendamento.service.js
async function cadastrar(dados) {
  const valor = calcularValor(dados.especie, dados.servico)
  if (!valor) throw criarErro('Serviço inválido', 400)
  return AgendamentoRepository.criar({ ...dados, valor })
}

// repositories/agendamento.repository.js
async function criar(dados) {
  return Agendamento.create(dados)
}
```

O service aplica a regra (o preço de cada serviço) e lança um erro quando algo não faz sentido. O repository só conversa com o banco.

Regra de ouro das camadas



Controller

Não fala com o banco.



Service

Não conhece **req**
nem **res**.



Repository

Não tem regra de negócio.

Se uma camada começa a fazer o trabalho da outra, o código volta a virar o `server.js` de antes.



Autenticação

Provar quem você é

Rotas públicas e protegidas



Públicas

- Qualquer um acessa.
- Ex.: cadastro e login.



Protegidas

- Só com um token válido.
- Ex.: ver e editar o perfil, cadastrar livros.

Autenticar dá controle de acesso, rastreabilidade (quem fez o quê) e segurança para os dados de cada usuário.

O login, passo a passo



1. Envia

O cliente manda e-mail e senha.

2. Confere

A API compara a senha com o hash (bcrypt).

3. Gera o token

Deu certo: a API cria um JWT.

4. Usa o token

O cliente envia o token nas rotas protegidas.



bcrypt

Senhas nunca são guardadas como texto

O que é hash?



Transformação de mão única

"123456" vira algo como "\$2b\$10\$k4P4...", e não dá para voltar.



Salvamos só o hash

A senha em texto nunca vai para o banco.



Comparar, não descriptografar

No login, o bcrypt calcula de novo e compara: confere ou não confere.



Se o banco vazar

Quem pegar os dados vê hashes, não as senhas dos usuários.

Cadastro: `bcrypt.hash()`

```
import bcrypt from 'bcryptjs'

const saltRounds = 10
const senhaHash = await bcrypt.hash(senha, saltRounds)

await Usuario.create({ nome, email, senhaHash })
```

`bcrypt.hash` → gera o hash da senha. É assíncrono: precisa de **`await`**.

`saltRounds` → o "custo": quanto maior, mais lento para gerar e para um atacante quebrar. 10 é um bom ponto de partida.

`senhaHash` → o nome do campo já deixa claro que ali não está a senha.

Login: bcrypt.compare()

```
const usuario = await Usuario.findOne({ email }).select('+senhaHash')
if (!usuario) {
  throw criarErro('E-mail ou senha incorretos.', 401)
}

const senhaCorreta = await bcrypt.compare(senha, usuario.senhaHash)
if (!senhaCorreta) {
  throw criarErro('E-mail ou senha incorretos.', 401)
}
```

A mensagem é a mesma nos dois casos: a API não entrega se o e-mail existe. Isso dificulta quem tenta descobrir contas cadastradas.



Cuidado: nunca devolva a senha nem o hash

Uma resposta com **senhaHash** entrega um pedaço da segurança do usuário.

- No Schema, use **select: false**: o campo não vem nas buscas, a não ser que você peça com **.select("+senhaHash")**.
- E remova o hash ao transformar o documento em JSON, como o boilerplate faz no Model.

```
senhaHash: { type: String, required: true, select: false }
```



JSON Web Token

Autenticação é como um evento



Login = bilheteria

Você mostra o documento e prova quem é.



Token = pulseira

Você recebe uma pulseira que vale por um tempo.



Middleware = segurança

Em cada área, alguém confere a pulseira.



Autorização = área VIP

Nem toda pulseira entra em todo lugar.

Gerando o token: `jwt.sign()`

```
import jwt from 'jsonwebtoken'

const token = jwt.sign(
  { id: usuario._id, email: usuario.email },
  process.env.JWT_SECRET,
  { expiresIn: '1d' }
)

res.json({ token })
```

payload → o que vai dentro do token: o id e o e-mail do usuário.

JWT_SECRET → a chave que assina o token. Fica no **.env**.

expiresIn → o prazo de validade. Depois disso, é preciso logar de novo.



Cuidado: o JWT não é criptografado

Qualquer pessoa consegue **ler** o conteúdo de um token (cole um em jwt.io). A assinatura só impede que ele seja **alterado**.

- Nunca coloque senha, hash ou dados sensíveis no payload: só o necessário, como o id.
- O **JWT_SECRET** precisa ser longo, aleatório e ficar no **.env**. Quem tem o secret consegue gerar tokens válidos.

```
# .env
JWT_SECRET=uma_frase_longa_e_aleatoria_que_ninguem_adivinha_3f9a1c
```

O middleware de autenticação

```
function autenticar(req, res, next) {  
  const authHeader = req.headers.authorization  
  if (!authHeader) {  
    return res.status(401).json({ message: 'Token não informado.' })  
  }  
  
  const [tipo, token] = authHeader.split(' ')  
  if (tipo !== 'Bearer' || !token) {  
    return res.status(401).json({ message: 'Formato inválido.' })  
  }  
  try {  
    req.usuario = jwt.verify(token, process.env.JWT_SECRET)  
    next()  
  } catch (erro) {  
    return res.status(401).json({ message: 'Token inválido.' })  
  }  
}
```

O `jwt.verify` confere a assinatura e a validade. Deu certo? O `payload` vai para `req.usuario` e o `next()` libera a rota.



Cuidado: token expirado ou inválido?

O `jwt.verify` lança erros diferentes, e dá para avisar o cliente do jeito certo.

- **TokenExpiredError**: o token era bom, mas venceu. O app deve mandar o usuário logar de novo.
- **JsonWebTokenError**: token alterado ou assinado com outro secret.

```
catch (erro) {  
  const expirou = erro.name === 'TokenExpiredError'  
  return res.status(401).json({ message: expirou ? 'Sessão expirada' : 'Token inválido' })  
}
```

Rota protegida na prática

Sem token

```
GET /api/usuarios/perfil
```

→ 401 Unauthorized

```
{ "message": "Token não informado." }
```

Com token

```
GET /api/usuarios/perfil
```

```
Authorization: Bearer eyJhbGci...
```

→ 200 OK

```
{ "nome": "Maria", "email": "maria@email.com" }
```

Na rota: `router.get("/api/usuarios/perfil", autenticar, UsuarioController.perfil)`. O middleware vem antes do controller.



Cuidado: o formato do header

O header precisa ser exatamente **Authorization: Bearer <token>**, com um espaço entre os dois.

- No Postman, use a aba **Authorization > Bearer Token** e cole só o token, sem aspas.
- Token expirado dá 401 mesmo estando certo: faça login de novo e copie o token novo.

Autenticação x autorização



Autenticação

Quem é você?

**Sem token, ou token inválido: 401
Unauthorized.**



Autorização

O que você pode fazer?

**Logado, mas sem permissão: 403
Forbidden.**

Exemplo: você está logado (autenticado), mas não pode apagar o livro de outra pessoa (não autorizado).



Validação e erros

Middlewarees que deixam os controllers limpos

Middleware de validação

```
function validarCampos(campos) {  
  return (req, res, next) => {  
    const faltando = campos.filter((campo) => !req.body[campo])  
    if (faltando.length > 0) {  
      const lista = faltando.join(', ')  
      return res.status(400).json({ message: `Faltou: ${lista}` })  
    }  
    next()  
  }  
}  
  
router.post(  
  '/api/auth/cadastro',  
  validarCampos(['nome', 'email', 'senha']),  
  AuthController.cadastrar  
)
```

Uma função que **cria um middleware**: cada rota diz quais campos exige, e o **controller já recebe os dados completos**.

Criando erros com status

```
// utils/criarErro.js
export function criarErro(mensagem, status) {
  const erro = new Error(mensagem)
  erro.status = status
  return erro
}

// no service
if (emailJaExiste) throw criarErro('E-mail já cadastrado.', 409)
```

O service não conhece o res, mas pode dizer qual status o erro merece.

Middleware de erro: um lugar só

```
// middlewares/erro.middleware.js
export function tratarErros(err, req, res, next) {
  res.status(err.status || 500).json({ message: err.message })
}

// no controller
catch (erro) {
  next(erro) // o middleware de erro responde
}

// no app.js, depois de todas as rotas
app.use(tratarErros)
```

Todo erro passa pelo `next(erro)` e cai num único lugar: as respostas de erro ficam padronizadas.



Autorização por papel

Admin e usuário comum: os dois logados, com permissões diferentes

O papel no Model do usuário

```
// src/models/usuario.model.js (dentro do Schema)
papel: {
  type: String,
  enum: ['usuario', 'admin'],
  default: 'usuario',
},
```

enum → o campo só aceita esses valores.

default → todo cadastro novo nasce usuário comum.

cadastro → o **auth.service** monta o usuário campo a campo: ninguém escolhe o próprio papel.

O papel dentro do token

auth.service.js (gerarToken)

```
const dadosDoToken = {  
  id: usuario._id.toString(),  
  email: usuario.email,  
  papel: usuario.papel,  
}
```

autenticacao.middleware.js

```
req.usuario = {  
  id: dadosDoToken.id,  
  email: dadosDoToken.email,  
  papel: dadosDoToken.papel,  
}
```

O boilerplate copia do token só o que estiver listado: o papel precisa entrar nos dois lados.

O middleware autorizar

```
// src/middlewares/autorizacao.middleware.js
import criarErro from '../utils/criarErro.js'

function autorizar(...papeisPermitidos) {
  return (req, res, next) => {
    if (!papeisPermitidos.includes(req.usuario.papel)) {
      return next(criarErro('Acesso negado.', 403))
    }
    return next()
  }
}

export default autorizar
```

...papeis → aceita um ou vários:

autorizar("admin") ou **autorizar("admin", "gerente")**.

return (req, res, next) → uma função que cria o middleware, como o **validarCampos**.

403 → sabemos quem é (autenticado), mas ele não tem permissão.

Uma rota só para admin

Usuário comum

```
GET /api/usuarios  
Authorization: Bearer (token de usuário)
```

```
→ 403 Forbidden  
{ "message": "Acesso negado." }
```

Admin

```
GET /api/usuarios  
Authorization: Bearer (token de admin)
```

```
→ 200 OK  
{ "usuarios": [ ... ] }
```

Na rota: `router.get("/", autenticar, autorizar("admin"),
UserController.listar)`



Cuidado: a ordem dos middlewares

Com o **autorizar** antes do **autenticar**, o **req.usuario** ainda não existe: o servidor quebra com **500**.

- Quem preenche o **req.usuario** é o **autenticar**. Ele vem sempre primeiro.
- Leia a rota da esquerda para a direita: é a ordem em que os middlewares rodam.

```
router.get('/', autorizar('admin'), autenticar, UsuarioController.listar) // 500
router.get('/', autenticar, autorizar('admin'), UsuarioController.listar) // certo
```



Cuidado: o papel nunca vem do body

Se o cadastro salvasse o `req.body` inteiro, bastaria mandar `"papel": "admin"` para virar administrador.

- O `auth.service` do boilerplate monta o usuário campo a campo (nome, e-mail e senhaHash): o papel fica no default.
- Mantenha assim, e faça o mesmo no `PATCH /perfil`: só nome e senha podem mudar.

```
// POST /api/auth/cadastro
{ "nome": "Ana", "email": "ana@email.com", "senha": "123456", "papel": "admin" }
// → Ana é criada com papel "usuario": o campo extra é ignorado
```

Como nasce o primeiro admin?

- 1 Cadastre-se normalmente**
POST /api/auth/cadastro: o usuário nasce comum.
- 2 Edite o documento no banco**
No Atlas (Browse Collections) ou no Compass, troque o campo para papel: "admin".
- 3 Faça login de novo**
Só o token novo carrega o papel de admin.

Depois disso, um admin pode promover outros usuários, se o sistema tiver essa rota (protegida com autorizar("admin")).



Cuidado: o token guarda o papel antigo

O papel entra no token **no login. Promoveu ou rebaixou alguém?
O token antigo continua com o papel antigo até vencer.**

- Quem foi promovido precisa fazer login de novo.
- Token gerado antes de o papel existir nem tem o campo: o **autorizar** responde 403.
- Precisa de efeito imediato? O middleware busca o usuário no banco a cada requisição: mais seguro, com uma consulta a mais.



Trocando a senha

Só o token não basta: peça a senha atual

Duas formas de trocar a senha

Boilerplate: PATCH /perfil

```
PATCH /api/usuarios/perfil  
Authorization: Bearer eyJ...
```

```
{ "senha": "novaSenha123" }
```

→ 200: senha trocada

Mais seguro: PATCH /senha

```
PATCH /api/usuarios/senha  
Authorization: Bearer eyJ...
```

```
{ "senhaAtual": "123456",  
  "novaSenha": "novaSenha123" }
```

→ 200: senha trocada

Só com o token, quem pegar um computador logado toma a conta. Pedir a senha atual prova que é o dono.

O service: trocarSenha

```
// services/usuario.service.js
async function trocarSenha(idDoUsuario, { senhaAtual, novaSenha }) {
  if (!senhaAtual || !novaSenha) {
    throw criarErro('Informe a senha atual e a nova senha.', 400)
  }
  validarSenha(novaSenha)

  const usuario = await UsuarioRepository.buscarPorIdComSenha(idDoUsuario)
  if (!usuario) throw criarErro('Usuário não encontrado.', 404)

  const confere = await bcrypt.compare(senhaAtual, usuario.senhaHash)
  if (!confere) throw criarErro('Senha atual incorreta.', 400)

  const saltRounds = Number(process.env.BCRYPT_SALT_ROUNDS || 10)
  const senhaHash = await bcrypt.hash(novaSenha, saltRounds)
  await UsuarioRepository.atualizarPorId(idDoUsuario, { senhaHash })
  return { message: 'Senha alterada com sucesso.' }
}
```

Senha atual errada: 400, e não 401. No frontend, 401 significa "sua sessão acabou, faça login" (Módulo 11).

Repository, rota e controller

```
// repositories/usuario.repository.js
async function buscarPorIdComSenha(id) {
  return Usuario.findById(id).select('+senhaHash')
}

// routes/usuario.routes.js
router.patch('/senha', autenticar, UsuarioController.trocarSenha)

// controllers/usuario.controller.js
async function trocarSenha(req, res, next) {
  try {
    const resultado = await UsuarioService.trocarSenha(req.usuario.id, req.body)
    return res.status(200).json(resultado)
  } catch (error) {
    return next(error)
  }
}
```

O `select("+senhaHash")` é necessário porque o Model tem `select: false`. E cada função nova entra no objeto exportado do arquivo.



Cuidado: o token antigo continua valendo

Trocar a senha **não invalida os tokens já emitidos: o JWT é conferido só pela assinatura e pela validade.**

- Por isso um **expiresIn** curto importa: limita o tempo de vida de um token vazado.
- Solução avançada: salvar **senhaAlteradaEm** na troca e recusar tokens emitidos antes disso. O **iat** do token diz quando ele foi criado, em segundos.

```
// avançado: no middleware, depois de buscar o usuário no banco
if (usuario.senhaAlteradaEm) {
  const alterouEm = Math.floor(usuario.senhaAlteradaEm.getTime() / 1000)
  if (dadosDoToken.iat < alterouEm) {
    return next(criarErro('Sessão expirada. Faça login de novo.', 401))
  }
}
```



Testando a segurança

Como testar a API no Postman

- 1 Cadastre um usuário**
POST /api/auth/cadastro com nome, e-mail e senha.
- 2 Faça login**
POST /api/auth/login e copie o **token** da resposta.
- 3 Acesse uma rota protegida**
Com Authorization > Bearer Token: deve dar 200.
- 4 Tente quebrar**
Sem token, com token errado e com senha errada: 401 em todos.

Checklist de segurança



Só o hash

O banco guarda apenas o **senhaHash**.



Nada de hash na resposta

select: false e remoção no JSON.



Erro genérico no login

"E-mail ou senha incorretos."



Secret no .env

E o **.env** no **.gitignore**.



Token com prazo

Sempre com **expiresIn**.



Rotas protegidas

O middleware em toda rota privada.



Atividade: Livros

Um recurso novo, protegido e com dono

Um livro pertence a alguém

```
// src/models/livro.model.js
const livroSchema = new mongoose.Schema(
  {
    titulo: { type: String, required: true, trim: true },
    autor: { type: String, required: true, trim: true },
    dono: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Usuario',
      required: true,
    },
  },
  { timestamps: true }
)
```

Diferente do usuário, o livro guarda o `_id` do dono: o mesmo relacionamento por referência do Módulo 8.

As rotas dos livros

Rota (protegida)	Service	O que garante
POST /api/livros	registrar(idDoUsuario, dados)	O dono é quem está logado
GET /api/livros	listarMeus(idDoUsuario)	Só os livros do usuário
GET /api/livros/:id	buscarMeu(idDoUsuario, id)	Livro de outro: 404
PATCH /api/livros/:id	atualizarMeu(idDoUsuario, id, dados)	Só altera os próprios
DELETE /api/livros/:id	removerMeu(idDoUsuario, id)	Só remove os próprios

Ordem sugerida: Model → Repository → Routes (registradas no app.js) → Service → Controller.



Cuidado: o dono vem do token, nunca do body

Se o dono vier do **req.body**, qualquer usuário cadastra ou lê livros **em nome de outro**.

- O id de quem está logado está em **req.usuario.id**, colocado lá pelo middleware.
- Toda busca filtra pelo dono: **Livro.findOne({ _id: id, dono: idDoUsuario })**. Livro de outra pessoa = não encontrado.

```
// no controller
const idDoUsuario = req.usuario.id
const livro = await LivroService.registrar(idDoUsuario, req.body)
```

Arquivos do recurso Livro

```
src/  
├── models/livro.model.js  
├── repositories/livro.repository.js  
├── services/livro.service.js  
├── controllers/livro.controller.js  
└── routes/livro.routes.js      (registrar no app.js)
```

Um arquivo por camada: o mesmo caminho da requisição que vimos no começo do módulo.

Atividade Prática

Livros

1. Crie o Model `livro.model.js`, com o campo `dono` referenciando o usuário.
2. Crie o repository, o service, o controller e o arquivo de rotas, e registre as rotas no `app.js`.
3. Implemente as cinco rotas protegidas da tabela, todas com o middleware `autenticar`.
4. Teste no Postman com dois usuários: um não pode ver nem alterar os livros do outro.



Dica: o id do dono é sempre `req.usuario.id`. Se precisar de uma consulta nova, ela nasce no repository.



Desafio: crie o papel `admin` e a rota `GET /api/usuarios` só para admins. **Bônus:** a troca de senha com a senha atual.

Lista de exercícios



Lista de código

Autenticação e MVC: bcrypt, JWT e camadas, em peças soltas.



APIs com o boilerplate

Biblioteca, Academia e Petshop.



Desafios

Finanças, Lions Bet e Tarefas: APIs completas com login.

No repositório: `modulo09/lista_de_exercicios/`

Desafio: flashcards com dono

Fio condutor

Leve o flashcards do Módulo 8 para o boilerplate e dê um dono a cada baralho.

- Crie o recurso Baralho arquivo por arquivo, como o Livro: model, repository, service, controller e rotas.
- No model, acrescente `usuario: ObjectId`, com `ref: "Usuario"` e obrigatório.
- Proteja as rotas `/api/baralhos` com o middleware autenticar.
- No cadastro, o dono vem de `req.usuario.id`; na listagem, busque só os baralhos desse usuário.
- Teste no Postman com dois usuários: um não vê nem apaga o baralho do outro.



Cuidado: o dono vem do token, nunca do body.



Desafio: faça o mesmo com o Flashcard: só o dono do baralho cria cartões nele.

Material do módulo



Aponte a câmera do celular

`github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo09`



conteudo/
teoria comentada e o cheat sheet do módulo



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

No Módulo 9B, a API ganha testes automatizados: nada de clicar em tudo no Postman a cada mudança.